上海交通大學

SHANGHAI JIAO TONG UNIVERSITY

KEM in Modern Protocol Design:

From CCA Security to Post-Quantum Deployment

Semester:	2025-2026-Spring
Members:	Shenyu Wu, Yuezhao Ma, Shengyuan Huang
School:	School of Computer Science
Major:	Information Security
Course:	NIS3352 Modern Cryptography II
Instructor:	Songsong Li
Date:	May 8th, 2026

Abstract

This paper examines the Key Encapsulation Mechanism (KEM) as the foundational key-establishment primitive for modern and post-quantum cryptographic protocols. We trace the theoretical evolution of KEM through key milestones: Shoup's KEM/DEM composition, the Cramer-Shoup paradigm establishing IND-CCA security as the baseline, and the Fujisaki-Okamoto (FO) transformation that generically lifts CPA-secure schemes to CCA security. Furthermore, we bridge theory and practice by presenting jkem, a teaching-oriented Rust implementation of the newly standardized ML-KEM (FIPS 203). Through empirical benchmarks and constant-time evaluations, we illustrate the engineering intricacies of module-lattice operations and implicit rejection mechanisms. Finally, we analyze the deployment of KEMs in real-world protocols like HPKE and hybrid TLS 1.3, highlighting that while the KEM abstraction provides a clean modular boundary, robust secure channels require careful integration with transcript binding, authentication, and downgrade protection.

Keywords: Key Encapsulation Mechanism (KEM), Post-Quantum Cryptography (PQC), Fujisaki-Okamoto Transformation, ML-KEM, Hybrid Key Exchange

Contents

Chapter 1 Introduction

Evolution of Key Establishment Paradigms: From Public-Key Encryption to Key Encapsulation Mechanism

Since the birth of public-key cryptography, its theoretical abstraction and engineering practice have continuously evolved through mutual promotion and compromise. Before exploring why modern secure protocols have comprehensively embraced the Key Encapsulation Mechanism (KEM), it is essential to review the semantics of classic key establishment primitives and the bottlenecks they encountered in real-world deployments.

1.1 Limitations of Classic Primitives: Diffie-Hellman and PKE

Early public-key cryptography sought to address two independent yet deeply intertwined challenges: establishing a shared secret over an insecure channel, and enabling non-interactive confidential communication. In 1976, Diffie and Hellman proposed their landmark key exchange protocol (DH) based on the discrete logarithm problem[1]. The elegance of DH lies in its use of homomorphic mathematical structures (e.g., $g^{xy} = (g^x)^y$), allowing communicating parties to "aggregate" a shared secret out of public interactions. Furthermore, when instantiated with ephemeral keys, DH naturally provides *forward secrecy*. However, its fundamental limitation is the strict requirement for interactive communication.

In parallel, general-purpose Public-Key Encryption (PKE), epitomized by RSA, offered an alternative paradigm. PKE functions as a digital "envelope"—the sender uses the receiver's public key to encrypt an arbitrary message, which the receiver then decrypts. Under traditional semantic definitions ($c \leftarrow \mathrm{Enc}(pk, m)$), a PKE scheme enables non-interactive encryption but typically lacks native forward secrecy unless augmented with protocol-level key rotations.

Moreover, these two paradigms diverge significantly in their security goals. DH is typically analyzed in the random oracle or generic group model, targeting Indistinguishability under Chosen Plaintext Attack (IND-CPA) style security for the resulting shared secret. Conversely,

robust PKE schemes are designed to achieve Indistinguishability under Chosen Ciphertext Attack (IND-CCA) for arbitrary message spaces. In real-world network protocols, using PKE to directly process large volumes of application data is computationally prohibitive. Consequently, early engineering practices (such as RSA key transport in early TLS) relegated PKE to a fallback role: encrypting a short "pre-master secret" before switching to symmetric encryption. While functional, the powerful syntax of PKE—supporting arbitrary plaintext input—was largely wasted and often became a fertile ground for side-channel and chosen-ciphertext attacks.

1.2 The Protocol-Centric Shift: KEM Abstraction and Engineering Synergy

To bridge the gap between theoretical primitives and actual protocol requirements, the cryptographic community began to redefine the boundaries of key establishment. The genuine requirement of modern secure channels is remarkably specific: protocols do not need to encrypt meaningful text using public-key algorithms; they merely need to synchronize a short, high-entropy random bitstring between two endpoints.

Motivated by this observation, the concept of KEM was formalized, notably through Shoup's formalization of the KEM/DEM (Data Encapsulation Mechanism) composition paradigm in 2001[2]. In the KEM syntax, the encapsulation algorithm $(K, c) \leftarrow \text{Encaps}(pk)$ strips the caller of the ability to input a specific plaintext. Instead, the algorithm internally generates a random shared secret K and outputs its encapsulation c.

This subtle semantic shift triggered profound architectural improvements in modern protocol design. Primarily, it provided exceptional modularity underpinned by formal security guarantees. The KEM/DEM composition theorem states that combining an IND-CCA secure KEM with an IND-CCA secure DEM straightforwardly yields an IND-CCA secure hybrid encryption scheme. In modern protocols like the Hybrid Public Key Encryption standard (HPKE, RFC 9180)[3], the KEM is solely responsible for outputting high-entropy secrets, the Key Derivation Function (KDF) securely binds this secret to communication transcripts and identities, and an Authenticated Encryption with Associated Data (AEAD) cipher manages the bulk data. This decoupling not only simplifies security proofs but also facilitates highly flexible cipher suite negotiations.

More importantly, KEM serves as the vital bridge to the post-quantum cryptography (PQC) era. Under the threat of quantum computing, classical DH and PKE algorithms will be rendered insecure. However, mainstream post-quantum public-key schemes—particularly lattice-based (e.g., LWE or NTRU), code-based, and isogeny-based constructions—often involve the deliberate introduction of mathematical noise. Forcing these noisy structures to perfectly encrypt a user-specified plaintext requires significant parameter redundancy and complex Error Correction Codes (ECC). In contrast, negotiating a noisy random value and aligning it via hashing into a shared key is highly natural and efficient.

Because the KEM syntax perfectly circumvents the requirement to "carry exact plaintexts," it resonates flawlessly with the internal mechanics of PQC primitives. This explains why the National Institute of Standards and Technology (NIST), in its newly released post-quantum standard FIPS 203 (ML-KEM)[4], completely abandoned the traditional PKE interface, standardizing KEM as the sole asymmetric paradigm. The transition from PKE to KEM is not merely a sign of cryptography maturing from theory to engineering; it is the fundamental architecture enabling a seamless migration to the post-quantum future.

1.3 Scope and Assumptions

This review assumes the reader's familiarity with foundational topics typically covered in a modern cryptography course. Specifically, we presume a working knowledge of semantic security definitions for public-key encryption, the Learning With Errors (LWE) and Short Integer Solution (SIS) problems underlying lattice-based cryptography.

Consequently, this paper does not re-introduce these underlying mathematical primitives from scratch. Instead, our scope is strictly focused on the KEM abstraction itself: its formal security properties, its transformational constructions (such as the Fujisaki-Okamoto transform for achieving CCA security), and its structural role in large-scale protocol deployment. Mathematical preliminaries are kept minimal, introducing only the requisite notations necessary to analyze KEM constructions and rigorously evaluate their security arguments.

Chapter 2 Preliminaries

KEM/DEM Definition and IND-CCA Security

This section establishes the formal foundations for the remainder of the paper. We define the syntax and security notions for Key Encapsulation Mechanisms (KEMs) and Data Encapsulation Mechanisms (DEMs). These primitives form the building blocks of hybrid public-key encryption.

2.1 Key Encapsulation Mechanism (KEM)

A Key Encapsulation Mechanism abstracts the asymmetric part of a hybrid encryption scheme. Unlike a full public-key encryption scheme that encrypts arbitrary messages, a KEM is designed to encapsulate a random symmetric key.

Definition 2.1 KEM Syntax. A Key Encapsulation Mechanism KEM = (KeyGen, Encaps, Decaps) consists of three probabilistic polynomial-time algorithms:

- **Key Generation:** $\text{KeyGen}(1^\lambda) \to (pk, sk)$. Given a security parameter λ (in unary), outputs a public encapsulation key pk and a secret decapsulation key sk. The key pair may also include public parameters.
- **Encapsulation:** $\text{Encaps}(pk) \to (K, \psi)$. Given the public key pk, outputs a symmetric key $K \in \{0,1\}^{\kappa(\lambda)}$ (where $\kappa(\lambda)$ is the key length) and a ciphertext ψ (a bit string).
- **Decapsulation:** $\text{Decaps}(sk, \psi) \to K$ or $\perp$. Given the secret key sk and a ciphertext ψ, outputs either a symmetric key K or a distinguished rejection symbol $\perp$ indicating that ψ is invalid.

Correctness. For all $\lambda \in \mathbb{N}$, all $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$, and all $(K, \psi) \leftarrow \text{Encaps}(pk)$, we require:

$$\Pr[\text{Decaps}(sk, \psi) = K] \geq 1 - \delta,$$

where $\delta = \text{negl}(\lambda)$. We also call this KEM to be δ-correct.

Note: In classical KEM constructions (e.g., RSA-KEM, ElGamal-KEM), correctness is typically deterministic and holds with probability 1. However, in many post-quantum KEMs—particularly lattice-based constructions like ML-KEM, the correctness guarantee is probabilistic due to compression, noise, or decoding errors.

2.2 Data Encapsulation Mechanism (DEM)

A Data Encapsulation Mechanism captures the symmetric-key part of hybrid encryption. It is essentially a symmetric encryption scheme designed for one-time use with a random key.

Definition 2.2 DEM Syntax. A Data Encapsulation Mechanism DEM = (Encrypt, Decrypt) consists of two deterministic polynomial-time algorithms:

- **Encryption:** $\text{Encrypt}(K, m) \to \chi$. Given a symmetric key $K \in \{0,1\}^{\kappa(\lambda)}$ and a message $m \in \{0,1\}^*$, outputs a ciphertext χ.
- **Decryption:** $\text{Decrypt}(K, \chi) \to m$ or $\perp$. Given a symmetric key K and a ciphertext χ, outputs either a message m or a rejection symbol $\perp$.

Correctness. For all λ, all keys $K \in \{0,1\}^{\kappa(\lambda)}$, and all messages m:

$$\text{Decrypt}(K, \text{Encrypt}(K, m)) = m.$$

Unlike KEMs, DEMs are typically deterministic and perfectly correct. In practice, a DEM is often built from an authenticated encryption scheme (e.g., AES-GCM) or from a composition of a symmetric encryption scheme and a message authentication code (MAC), as in the DEM1 construction proposed by Shoup[2].

2.3 IND-CCA Security for KEM

We recall the standard indistinguishability under adaptive chosen ciphertext attack (IND-CCA) security notion for KEMs[2]. Here "adaptive CCA" means that the adversary may continue making decapsulation queries even after receiving the challenge ciphertext, subject only to the restriction that it cannot query the exact challenge ciphertext itself.

Definition 2.3 KEM IND-CCA Game. Let KEM = (KeyGen, Encaps, Decaps) be a KEM. For an adversary A, the IND-CCA experiment $\mathrm{Exp}^{\text{ind-cca}}_{\text{KEM},A}(1^\lambda)$ proceeds as follows:

1. **Setup:** Compute $(pk, sk) \leftarrow \mathrm{KeyGen}(1^\lambda)$. Give pk to A.
2. **Pre-Challenge Decryption Queries:** A may query a decapsulation oracle $\mathrm{Decaps}(sk, \cdot)$ on arbitrary ciphertexts ψ. The oracle returns $\mathrm{Decaps}(sk, \psi)$ (which may be $\perp$).
3. **Challenge:** A signals it is ready. The challenger computes:
   - $(K_0, \psi^*) \leftarrow \mathrm{Encaps}(pk)$
   - $K_1 \leftarrow \{0,1\}^{\kappa(\lambda)}$ uniformly at random
   - Choose $b \leftarrow \{0,1\}$ uniformly

   Give (ψ^*, K_b) to A.
4. **Post-Challenge Decryption Queries:** A may continue to query $\mathrm{Decaps}(sk, \cdot)$, subject to the restriction that $\psi \neq \psi^*$.
5. **Output:** A outputs a guess $b' \in \{0,1\}$. The experiment outputs 1 if $b' = b$, and 0 otherwise.

Definition 2.4 KEM IND-CCA Advantage. The advantage of A against KEM is:

$$\mathrm{Adv}^{\text{ind-cca}}_{\text{KEM}}(A) = \left| \Pr[\mathrm{Exp}^{\text{ind-cca}}_{\text{KEM},A} = 1] - \frac{1}{2} \right|.$$

We say KEM is **IND-CCA secure** if for all probabilistic polynomial-time adversaries A, $\mathrm{Adv}^{\text{ind-cca}}_{\text{KEM}}(A) \leq \mathrm{negl}(\lambda)$.

2.4 IND-CCA Security for DEM

For DEMs, we require a notion of IND-CCA security that captures the one-time use of the symmetric key in hybrid encryption. This notion is essentially the standard IND-CCA for symmetric encryption, but without encryption queries (since the key is used only once).

Definition 2.5 DEM IND-CCA Game. Let DEM = (Encrypt, Decrypt) be a DEM with key length $\kappa(\lambda)$. For an adversary B, the experiment $\mathrm{Exp}^{\text{ind-cca}}_{\text{DEM},B}(1^\lambda)$ proceeds as follows:

1. **Key Generation:** Choose $K \leftarrow \{0,1\}^{\kappa(\lambda)}$ uniformly at random.
2. **Find Phase:** B outputs two messages $m_0, m_1 \in \{0,1\}^*$ of equal length.
3. **Challenge:** Choose $b \leftarrow \{0,1\}$ uniformly. Compute $\chi^* \leftarrow \mathrm{Encrypt}(K, m_b)$. Give χ^* to B.

4. **Decryption Queries:** B may query a decryption oracle $\text{Decrypt}(K, \cdot)$ on arbitrary ciphertexts $\chi \neq \chi^*$. The oracle returns $\text{Decrypt}(K, \chi)$ (which may be $\perp$).
5. **Output:** B outputs a guess $b' \in \{0, 1\}$. The experiment outputs 1 if $b' = b$, and 0 otherwise.

Definition 2.6 DEM IND-CCA Advantage. The advantage of B against DEM is:

$$\text{Adv}^{\text{ind-cca}}_{\text{DEM}}(B) = \left| \Pr[\text{Exp}^{\text{ind-cca}}_{\text{DEM},B} = 1] - \frac{1}{2} \right|.$$

We say DEM is **IND-CCA secure** if for all probabilistic polynomial-time adversaries B, $\text{Adv}^{\text{ind-cca}}_{\text{DEM}}(B) \leq \text{negl}(\lambda)$.

Chapter 3 Technical Development

Key Milestones in KEM Design and Post-Quantum Standardization

This section will present the technical development, starting with Shoup's KEM/DEM composition theorem, then the Cramer-Shoup scheme, the Fujisaki-Okamoto transformation, and finally ML-KEM as a post-quantum instantiation.

3.1 Milestone 1: Shoup's KEM/DEM Systematization

Shoup's seminal work[2] systematized the division between key encapsulation and data encapsulation, formally defining the KEM/DEM paradigm. The key insight is that cryptographers can independently study KEM and DEM; their composition yields a hybrid encryption system with composable security properties. Specifically, if a KEM satisfies IND-CCA security and a DEM satisfies IND-CCA security (in the sense of symmetric encryption with ciphertext integrity), then the resulting hybrid scheme achieves IND-CCA security. This modularity mirrors engineering best practices and has influenced standardization efforts including ISO/IEC 18033-2.

Definition 3.1 KEM/DEM Paradigm. Given a KEM and a compatible DEM (i.e., with matching key lengths), one can construct a hybrid public-key encryption scheme PKE = Hybrid[KEM, DEM] as follows:

- **Key Generation:** Same as KEM.KeyGen$(1^\lambda) \to (pk, sk)$.
- **Encryption:** To encrypt a message m under public key pk:
  1. $(K, \psi) \leftarrow$ KEM.Encaps(pk)
  2. $\chi \leftarrow$ DEM.Encrypt(K, m)
  3. Output ciphertext $C = (\psi, \chi)$
- **Decryption:** To decrypt a ciphertext $C = (\psi, \chi)$ under secret key sk:
  1. $K \leftarrow$ KEM.Decaps(sk, ψ); if $K = \perp$, output $\perp$
  2. $m \leftarrow$ DEM.Decrypt(K, χ); output m (or $\perp$ if decryption fails)

Theorem 3.2 KEM/DEM Composition. If KEM is IND-CCA secure and DEM is IND-CCA secure, then the hybrid scheme PKE is IND-CCA secure.

Proof. Let A be an adversary against the hybrid scheme. Define game $\mathbf{G}_0$ as the real IND-CCA game. We construct a sequence of games:

- **Game $\mathbf{G}_1$:** Replace the real KEM key K^* in the challenge ciphertext with a uniformly random key K' of the same length. The difference between $\mathbf{G}_0$ and $\mathbf{G}_1$ is exactly a KEM IND-CCA game, bounded by $\mathrm{Adv}^{\text{ind-cca}}_{\text{KEM}}(B_1)$ for a suitable adversary B_1 against KEM.
- **Game $\mathbf{G}_2$:** Replace the DEM encryption of m_b under K' with an encryption of a fixed dummy message (e.g., all zeros). The difference between $\mathbf{G}_1$ and $\mathbf{G}_2$ is exactly a DEM IND-CCA game, bounded by $\mathrm{Adv}^{\text{ind-cca}}_{\text{DEM}}(B_2)$.
- In $\mathbf{G}_2$, the challenge ciphertext is independent of b, so A's advantage is 0.

Next we explicitly construct the adversaries B_1 and B_2.

B_1 is a reduction to the KEM IND-CCA game.

- **Input:** B_1 receives a KEM public key pk_{kem} and access to a KEM decapsulation oracle $\mathrm{Decaps}_{\text{kem}}(\cdot)$.
- **Setup:** B_1 generates the DEM parameters locally and defines the hybrid public key as $pk = (\mathrm{pk}_{\text{kem}}, \mathrm{params}_{\text{dem}})$.
- **Decryption queries:** On each query (ψ, χ) from A, B_1 forwards ψ to its KEM decapsulation oracle. If the oracle returns $K = \perp$, B_1 returns $\perp$ to A; otherwise it returns $\mathrm{DEM.Decrypt}(K, \chi)$.
- **Challenge:** When A submits messages m_0, m_1, B_1 forwards the request to the KEM challenger and receives (ψ^*, K_b), where $b \in \{0, 1\}$. It chooses a random bit β, computes $\chi^* = \mathrm{DEM.Encrypt}(K_b, m_\beta)$, and returns (ψ^*, χ^*) to A.
- **Restrictions:** If A later queries a ciphertext whose first component equals ψ^*, B_1 must refuse the query because the KEM game forbids asking for decapsulation of the challenge ciphertext.
- **Output:** When A outputs a guess β', B_1 outputs 1 if $\beta' = \beta$, and 0 otherwise.

The simulation is perfect when $b = 0$ because K_b is the real KEM key. When $b = 1$, the challenge key K_b is uniform and independent of the challenge messages, so B_1 simulates the

hybrid game where the DEM key is random. Thus

$$\left|\Pr[\beta' = \beta] - \frac{1}{2}\right| \leq \mathrm{Adv}^{\text{ind-cca}}_{\mathrm{KEM}}(B_1).$$

B_2 is a reduction to the DEM IND-CCA game.

- **Input:** B_2 receives a DEM challenge oracle that holds a hidden key K^* and answers decryption queries on ciphertexts $\chi \neq \chi^*$.
- **Setup:** B_2 generates a fresh KEM key pair $(\mathrm{pk}_{\mathrm{kem}}, \mathrm{sk}_{\mathrm{kem}})$ locally, and gives A the hybrid public key $pk = (\mathrm{pk}_{\mathrm{kem}}, \mathrm{params}_{\mathrm{dem}})$.
- **Decryption queries before challenge:** On each query (ψ, χ), B_2 computes $K = \mathrm{KEM.Decaps}(\mathrm{sk}_{\mathrm{kem}}, \psi)$. If $K = \perp$, it returns $\perp$. Otherwise it forwards χ to its DEM decryption oracle and returns the result.
- **Challenge:** When A submits m_0, m_1, B_2 runs $(\psi^*, K^*) \leftarrow \mathrm{KEM.Encaps}(\mathrm{pk}_{\mathrm{kem}})$ locally. It forwards (m_0, m_1) to the DEM challenger, receives the challenge ciphertext $\chi^* = \mathrm{DEM.Encrypt}(K^*, m_b)$ for random b, and returns (ψ^*, χ^*) to A.
- **Decryption queries after challenge:** If A queries (ψ, χ) with $\psi = \psi^*$ and $\chi \neq \chi^*$, B_2 forwards χ to the DEM decryption oracle because it knows the hidden key is K^*. If $\psi \neq \psi^*$, it decapsulates with $\mathrm{sk}_{\mathrm{kem}}$ and forwards the resulting key to the DEM oracle as before.
- **Output:** When A outputs a guess β', B_2 forwards β' as its output.

This simulation is also perfect: the hybrid ciphertext uses a KEM component generated exactly as in the real scheme, and the DEM component is provided by the DEM challenger under key K^*. Therefore the only difference between the real and ideal worlds is the DEM challenge encryption itself, giving

$$\left|\Pr[\beta' = b] - \frac{1}{2}\right| \leq \mathrm{Adv}^{\text{ind-cca}}_{\mathrm{DEM}}(B_2).$$

Combining the two reductions yields

$$\mathrm{Adv}_{\mathrm{PKE}}(A) \leq \mathrm{Adv}^{\text{ind-cca}}_{\mathrm{KEM}}(B_1) + \mathrm{Adv}^{\text{ind-cca}}_{\mathrm{DEM}}(B_2),$$

so the hybrid scheme is IND-CCA secure when both components are.

3.2 Milestone 2: The Cramer-Shoup KEM (CS3)

The Cramer-Shoup encryption scheme[5] was the first practical public-key encryption scheme provably secure against adaptive chosen ciphertext attacks (IND-CCA) without random oracles, under the Decisional Diffie-Hellman (DDH) assumption. While the original CS paper presented a full PKE, it also introduced a KEM called **CS3**[5] (and its most efficient variant **CS3b**), which we describe here.

3.2.1 Definition of Basic Assumptions

Definition 3.3 Decisional Diffie-Hellman (DDH) Assumption. Let $G \leftarrow \mathcal{G}(1^\lambda)$ be a cyclic group of prime order q with generator g. For any probabilistic polynomial-time adversary A, define

$$\mathrm{Adv}^{\mathrm{DDH}}_{\mathcal{G}}(A) = \left|\Pr[A(g, g^x, g^y, g^{xy}) = 1] - \Pr[A(g, g^x, g^y, g^z) = 1]\right|,$$

where $x, y, z \leftarrow \mathbb{Z}_q$ are uniformly random. The DDH assumption states that $\mathrm{Adv}^{\mathrm{DDH}}_{\mathcal{G}}(A) \leq \mathrm{negl}(\lambda)$ for all PPT adversaries A.

Definition 3.4 Target Collision Resistant (TCR) Assumption. Let HF be a keyed hash family with public key space $\mathcal{K}$ and range $\mathbb{Z}_q$. For any PPT adversary A, define

$$\mathrm{Adv}^{\mathrm{TCR}}_{\mathrm{HF}}(A) = \Pr\left[\begin{array}{c} hk \leftarrow \mathcal{K} \\ y \leftarrow \mathbb{Z}_q \\ (x, x') \leftarrow A(hk, y) \end{array} : x \neq x' \ \wedge \ \mathrm{HF}_{hk}(x) = \mathrm{HF}_{hk}(x') = y\right].$$

We say HF is target collision resistant if $\mathrm{Adv}^{\mathrm{TCR}}_{\mathrm{HF}}(A) \leq \mathrm{negl}(\lambda)$ for all PPT adversaries A.

Definition 3.5 Key Derivation Function (KDF) Security. Let KDF be a keyed key-derivation function with key space $\mathcal{K}_{\mathrm{dk}}$ and output length $\kappa(\lambda)$. For any PPT adversary A, define

$$\mathrm{Adv}^{\mathrm{dist}}_{\mathrm{KDF}}(A) = \left|\Pr\left[\tau = 1 : \begin{array}{c} \mathrm{dk} \leftarrow \mathcal{K}_{\mathrm{dk}} \\ a, b \leftarrow G \\ K \leftarrow \mathrm{KDF}_{\mathrm{dk}}(a, b) \\ \tau \leftarrow A(\mathrm{dk}, a, K) \end{array}\right] - \Pr\left[\tau = 1 : \begin{array}{c} \mathrm{dk} \leftarrow \mathcal{K}_{\mathrm{dk}} \\ a \leftarrow G \\ K \leftarrow \{0,1\}^{\kappa(\lambda)} \\ \tau \leftarrow A(\mathrm{dk}, a, K) \end{array}\right]\right|.$$

We say KDF is secure if $\mathrm{Adv}^{\mathrm{dist}}_{\mathrm{KDF}}(A) \leq \mathrm{negl}(\lambda)$ for all PPT adversaries A.

3.2.2 Syntax of CS3 KEM

Definition 3.6 Cramer-Shoup KEM. Let G be a cyclic group of prime order q with generator g. Let $H : G^2 \to \mathbb{Z}_q$ be a target collision resistant hash function. Let $\mathrm{KDF} : G^2 \to \{0,1\}^{\kappa(\lambda)}$ be a key derivation function.

- **Key Generation:** Choose random $x_1, x_2, y_1, y_2, z_1, z_2, w \leftarrow \mathbb{Z}_q$ with $w \neq 0$. Compute:

$$\hat{g} = g^w, \quad e = g^{x_1}\hat{g}^{x_2}, \quad f = g^{y_1}\hat{g}^{y_2}, \quad h = g^{z_1}\hat{g}^{z_2}.$$

  The public key is $pk = (\hat{g}, e, f, h)$, and the secret key is $sk = (x_1, x_2, y_1, y_2, z_1, z_2)$.
- **Encapsulation:** On input pk, choose a random $u \leftarrow \mathbb{Z}_q$. Compute:

$$a = g^u, \quad \hat{a} = \hat{g}^u, \quad b = h^u, \quad v = H(a, \hat{a}), \quad d = e^u f^{uv}.$$

  Output the ciphertext $\psi = (a, \hat{a}, d)$ and the key $K = \mathrm{KDF}(a, b)$.
- **Decapsulation:** On input sk and ciphertext $\psi = (a, \hat{a}, d)$:
  1. Verify that $a, \hat{a}, d \in G$. If not, reject.
  2. Compute $v = H(a, \hat{a})$.
  3. Check whether $d = a^{x_1+y_1 v} \cdot \hat{a}^{x_2+y_2 v}$. If not, reject.
  4. Compute $b = a^{z_1}\hat{a}^{z_2}$.
  5. Output $K = \mathrm{KDF}(a, b)$.

3.2.3 Correctness of CS3 KEM

Theorem 3.7 Correctness of CS3. For any honestly generated key pair (pk, sk) and any encapsulation randomness u:

$$\mathrm{Decaps}(sk, \mathrm{Encaps}(pk)) = \mathrm{KDF}(a, b),$$

where $a = g^u$ and $b = h^u$.

Proof. From the encapsulation, we have $a = g^u$, $\hat{a} = \hat{g}^u = g^{wu}$. The decapsulation step first recomputes $b = a^{z_1}\hat{a}^{z_2} = g^{uz_1} \cdot g^{wuz_2} = g^{u(z_1+wz_2)} = h^u$, since $h = g^{z_1}\hat{g}^{z_2} = g^{z_1+wz_2}$.

For the verification step, note that $e = g^{x_1+wx_2}$ and $f = g^{y_1+wy_2}$. Then:

$$e^u f^{uv} = g^{u(x_1+wx_2)} \cdot g^{uv(y_1+wy_2)} = g^{u(x_1+y_1 v)+wu(x_2+y_2 v)}.$$

On the other hand, $a^{x_1+y_1 v} \cdot \hat{a}^{x_2+y_2 v} = g^{u(x_1+y_1 v)} \cdot g^{wu(x_2+y_2 v)}$, which is identical. Hence the verification passes, and decapsulation returns the same K as encapsulation.

3.2.4 IND-CCA Security of CS3 KEM

Theorem 3.8 IND-CCA Security of CS3. If the DDH assumption holds in G, H is a TCR hash function, and the KDF is secure as defined in Section 2.5, then the CS3 KEM is IND-CCA secure in the standard model.

Proof. Let A be an adversary making at most q_D decapsulation queries. We construct a sequence of games:

Game $\mathbf{G}_0$ (The standard game). The adversary receives pk, makes decapsulation queries, then receives a challenge (ψ^*, K_β) where $\beta \in \{0, 1\}$ is random, and outputs a guess β'. Let T_i be the event that $\beta' = \beta$ in $\mathbf{G}_i$.

Game $\mathbf{G}_1$ (Remove dependence on e, f, h). Modify the encapsulation oracle to compute $b \leftarrow a^{z_1}\hat{a}^{z_2}$ and $d \leftarrow a^{x_1+y_1v}\hat{a}^{x_2+y_2v}$, directly using the secret key instead of the public parameters e, f, h. This change is purely conceptual; the distribution of the challenge ciphertext is identical. So $\Pr[T_1] = \Pr[T_0]$.

Game $\mathbf{G}_2$ (Replace the DH triple). Modify the encapsulation oracle to sample $\hat{u} \leftarrow \mathbb{Z}_q$ independently and set $\hat{a} = \hat{g}^{\hat{u}}$. $(\hat{g} = g^w, a = g^u, \hat{a} = \hat{g}^u = g^{wu})$ in $\mathbf{G}_1$ and $(\hat{g} = g^w, a = g^u, \hat{a} = g^{w\hat{u}})$ in $\mathbf{G}_2$ form a DDH instance except $w \neq 0$, so by the DDH assumption, there is a reduction A_1 such that:

$$|\Pr[T_2] - \Pr[T_1]| \leq \mathrm{Adv}^{\mathrm{DDH}}_{\mathcal{G}}(A_1) + \frac{2}{q}.$$

Game $\mathbf{G}_3$ (Hash-collision rejection). Add a rule that if the adversary submits $(a, \hat{a}, d)$ with $(a, \hat{a}) \neq (a^*, \hat{a}^*)$ but $H(a, \hat{a}) = H(a^*, \hat{a}^*)$, the decapsulation oracle rejects immediately. Let R_3 be the event that a query is rejected in $\mathbf{G}_3$ but would have been accepted in $\mathbf{G}_2$. By the TCR assumption, there is a reduction A_2 such that:

$$\Pr[R_3] \leq \mathrm{Adv}^{\mathrm{TCR}}_{\mathrm{HF}}(A_2).$$

Games $\mathbf{G}_2$ and $\mathbf{G}_3$ are identical until R_3, so:

$$|\Pr[T_3] - \Pr[T_2]| \leq \Pr[R_3].$$

Game $\mathbf{G}_4$ (Exclude $\hat{u} = u$). We again modify the encapsulation oracle to sample $\hat{u} \leftarrow \mathbb{Z}_q \setminus \{u\}$ independently and set $\hat{a} = \hat{g}^{\hat{u}}$. It is clear that:

$$|\Pr[T_4] - \Pr[T_3]| \leq \frac{1}{q}.$$

Game $\mathbf{G}_5$ (Reject non-DH ciphertexts). Modify the decapsulation oracle to reject any ciphertext where $(a, \hat{a})$ is not a valid Diffie-Hellman pair, i.e., $\hat{a} \neq a^w$. Then in decryption $b = a^z$ and $d = a^{x+yv}$, where $x = x_1 + x_2 w$, $y = y_1 + y_2 w$, $z = z_1 + z_2 w$, and thus the decapsulation oracle no longer requires the individual values $x_1, x_2, y_1, y_2, z_1, z_2$, but only the aggregated values x, y, z. Let R_5 be the event that this rule triggers. Same as $\mathbf{G}_3$,

$$|\Pr[T_5] - \Pr[T_4]| \leq \Pr[R_5].$$

Using a linear algebra argument (Lemma 6.6 in the work of Cramer et al.[5]), one shows:

$$\Pr[R_5] \leq \frac{q_D(\lambda)}{q}.$$

Game $\mathbf{G}_6$ (Randomize b). Replace the computation of b in encryption with an independently random group element $b \leftarrow G$. By the Leftover Hash Lemma,

$$\begin{pmatrix} z \\ \log_g b \end{pmatrix} = \begin{pmatrix} 1 & w \\ u & w\hat{u} \end{pmatrix} \cdot \begin{pmatrix} z_1 \\ z_2 \end{pmatrix}$$

is uniformly random given the adversary's view, so the distribution of $b = a^{z_1}\hat{a}^{z_2}$ in $\mathbf{G}_5$ is statistically identical to that in $\mathbf{G}_6$. Thus,

$$\Pr[T_6] = \Pr[T_5].$$

Game $\mathbf{G}_7$ (Random KDF output). Replace $K = \mathrm{KDF}(a, b)$ in the challenge with a uniformly random string of the same length. By KDF security, there is a reduction A_3 such that:

$$|\Pr[T_7] - \Pr[T_6]| \leq \mathrm{Adv}^{\mathrm{dist}}_{\mathrm{KDF}}(A_3).$$

In $\mathbf{G}_7$, the challenge key is independent of the hidden bit β, so $\Pr[T_7] = 1/2$.

Final bound. Combining the game transitions gives:

$$\mathrm{Adv}^{\text{ind-cca}}_{\mathrm{CS3},A} \leq \mathrm{Adv}^{\mathrm{DDH}}_{\mathcal{G}}(A_1) + \mathrm{Adv}^{\mathrm{TCR}}_{\mathrm{HF}}(A_2) + \mathrm{Adv}^{\mathrm{dist}}_{\mathrm{KDF}}(A_3) + \frac{q_D(\lambda) + 3}{q}.$$

So under the assumptions above, CS3 KEM is IND-CCA secure.

3.2.5 The Efficient Variant: CS3b KEM

In practice, the most efficient version of the Cramer-Shoup KEM is **CS3b** (Section 9.3 of the article[5]), which simplifies key generation and decapsulation:

- **KeyGen:** Choose $x_1, x_2, y_1, y_2, z \leftarrow \mathbb{Z}_q$ and set $h = g^z$ (instead of $g^{z_1}\hat{g}^{z_2}$).
- **Encapsulation:** Unchanged.
- **Decapsulation:** After verifying d, compute $b = a^z$ directly, using the fact that $z = z_1 + wz_2$ can be aggregated.

This reduces decapsulation to three exponentiations (or two with precomputation) and eliminates the need for $\hat{a}$ in the KDF input.

The security proof for CS3b is almost identical to that of CS3, with an additional bound of q_D/q to account for the missing subgroup test. For details, please refer to the Theorem 9.2 in original paper[5].

3.3 Milestone 3: The Fujisaki-Okamoto Transformation and Its Modular Analysis

While the Cramer-Shoup KEM achieves IND-CCA security under the DDH assumption without random oracles, its design relies on a specific algebraic structure—namely, groups where the DDH problem is hard. This raises a natural question: can an IND-CCA secure KEM be built from *any* IND-CPA secure public-key encryption scheme, even those without such algebraic properties?

The Fujisaki-Okamoto (FO) transformation[6] answers this question affirmatively. It provides a generic, efficient method to convert a weak asymmetric encryption scheme (secure only against passive attacks) into a strong KEM (secure against adaptive chosen ciphertext attacks), in the random oracle model. This transformation has become a cornerstone of modern KEM design, particularly for post-quantum cryptography, where many low-level primitives naturally achieve only IND-CPA security.

3.3.1 The Underlying PKE Primitive

Before presenting the FO transformation, we must define the asymmetric encryption primitive it operates on and the security notions it assumes.

Definition 3.9 Public-Key Encryption Syntax. A public-key encryption scheme PKE = (KeyGen, Encrypt, Decrypt) consists of three algorithms:

- $\text{KeyGen}(1^\lambda) \to (pk, sk)$: Generates a public key pk and a secret key sk.
- $\text{Encrypt}(pk, m; r) \to c$: Given pk, a message m, and **coins** r (randomness), outputs a ciphertext c.
- $\text{Decrypt}(sk, c) \to m$ or $\perp$: Given sk and a ciphertext c, outputs either a message m or a rejection symbol $\perp$.

The Role of Coins. The term "coins" (or "random coins") refers to the internal randomness used by a probabilistic encryption algorithm. Since Encrypt is probabilistic, encrypting the same message twice with the same public key should produce different ciphertexts with overwhelming probability. This randomness is essential for achieving semantic security: without it, an adversary could distinguish encryptions of two messages by comparing ciphertexts.

Formally, let $\mathcal{R}$ denote the **coin space** of Encrypt. For a given pk, message m, and coins $r \in \mathcal{R}$, the encryption is deterministic: $c = \text{Encrypt}(pk, m; r)$. When we write $\text{Encrypt}(pk, m)$ without specifying r, it is understood that r is sampled uniformly from $\mathcal{R}$.

Definition 3.10 OW-CPA: One-Wayness under Chosen Plaintext Attack. The weakest standard security notion for PKE is **one-wayness** under chosen plaintext attack (OW-CPA). Informally, OW-CPA guarantees that an adversary who sees the encryption of a random message cannot recover the entire message. Formally, for an adversary A, define the OW-CPA advantage:

$$\text{Adv}^{\text{ow-cpa}}_{\text{PKE}}(A) = \Pr\left[m' = \text{Decrypt}(sk, c) : \begin{array}{c} (pk, sk) \leftarrow \text{KeyGen}(1^\lambda); \\ m \leftarrow \mathcal{M}; \\ c \leftarrow \text{Encrypt}(pk, m); \\ m' \leftarrow A(pk, c) \end{array} \right].$$

We say PKE is **OW-CPA secure** if for all PPT adversaries A, $\text{Adv}^{\text{ow-cpa}}_{\text{PKE}}(A) \leq \text{negl}(\lambda)$.

OW-CPA is strictly weaker than IND-CPA. Any deterministic encryption scheme (e.g., textbook RSA) that is one-way is a counterexample: such a scheme can be OW-CPA secure but not

IND-CPA secure (since determinism allows distinguishing encryptions of different messages). The FO transformation actually requires OW-CPA as its minimal assumption, though it can also start from IND-CPA with tighter parameters.

Definition 3.11 OW-PCVA: One-Wayness under Plaintext and Validity Checking Attacks. A stronger notion than OW-CPA is **one-wayness under plaintext and validity checking attacks** (OW-PCVA). The OW-PCVA game is similar to the OW-CPA game, except that the adversary additionally has access to a **plaintext checking oracle** $\mathrm{PCO}(m, c)$ and a **ciphertext validity checking oracle** $\mathrm{CVO}(c)$. Formally, for an adversary A, define the OW-PCVA advantage:

$$\mathrm{Adv}^{\text{ow-pcva}}_{\mathrm{PKE}}(A) = \Pr\left[\mathrm{PCO}(m', c^*) : \begin{array}{c}(pk, sk) \leftarrow \mathrm{KeyGen}(1^\lambda);\\ m^* \leftarrow \mathcal{M};\\ c^* \leftarrow \mathrm{Encrypt}(pk, m^*);\\ m' \leftarrow A^{\mathrm{PCO},\mathrm{CVO}}(pk, c^*)\end{array}\right],$$

where

$$\mathrm{PCO}(m \in \mathcal{M}, c) = [\mathrm{Decrypt}(sk, c) = m], \quad \mathrm{CVO}(c \neq c^*) = [\mathrm{Decrypt}(sk, c) \in \mathcal{M}].$$

We say PKE is **OW-PCVA secure** if for all PPT adversaries A, $\mathrm{Adv}^{\text{ow-pcva}}_{\mathrm{PKE}}(A) \leq \mathrm{negl}(\lambda)$. Restricting the adversary to PCO/CVO oracles yields the security notions OW-PCA and OW-VA, respectively.

OW-PCVA is strictly stronger than OW-CPA, as the plaintext checking oracle provides additional power to the adversary. We will see in Section 3.3.3 that the FO transformation's T component lifts OW-CPA to OW-PCVA via derandomization and re-encryption.

3.3.2 The Classical Fujisaki-Okamoto Transformation

The original FO transformation[6] converts a one-way secure (OW-CPA) public-key encryption scheme PKE into a CCA-secure KEM in the random oracle model. It consists of two hash functions G and H and proceeds as follows.

Definition 3.12 Classical FO KEM. Let $\mathrm{PKE} = (\mathrm{KeyGen}, \mathrm{Encrypt}, \mathrm{Decrypt})$ be an asymmetric encryption scheme with message space $\mathcal{M}$ and coin space $\mathcal{R}$. Let $G : \mathcal{M} \to \mathcal{R}$ and $H : \mathcal{M} \to \{0,1\}^\kappa$ be hash functions, modeled as random oracles. The FO-transformed KEM is defined as follows:

- **Key Generation:** Same as PKE.KeyGen$(1^\lambda) \to (pk, sk)$.
- **Encapsulation:** On input pk:
  1. Sample a random message $m \leftarrow \mathcal{M}$.
  2. Compute randomness $r = G(m)$.
  3. Compute ciphertext $c = \text{Encrypt}(pk, m; r)$.
  4. Compute symmetric key $K = H(m)$.
  5. Output (K, c).
- **Decapsulation:** On input sk and ciphertext c:
  1. Compute $m' = \text{Decrypt}(sk, c)$. If $m' = \perp$, output $\perp$.
  2. Compute $r' = G(m')$.
  3. If $\text{Encrypt}(pk, m'; r') = c$, output $K = H(m')$; otherwise output $\perp$.

Correctness follows directly from the correctness of PKE: for honestly generated ciphertexts, decryption yields the original m, recomputed encryption matches c, and K is recovered.

Theorem 3.13 Classical FO Security. If PKE is OW-CPA secure and G, H are modeled as random oracles, then the FO KEM is IND-CCA secure.

The essential idea of the proof is that any valid decapsulation query that differs from the challenge ciphertext must involve either a collision in G or H, or yield an m that inverts the challenge ciphertext—which would break one-wayness. While the classical FO transformation is conceptually elegant, its security analysis involves non-tight reductions and imposes perfect correctness requirements that are not always satisfied by modern lattice-based schemes, which often have a small but non-zero decryption error probability.

3.3.3 The Modular FO Framework

A major advance came from Hofheinz, Hövelmanns, and Kiltz[7], who provided a **modular analysis** of the Fujisaki-Okamoto transformation and its variants. Their work decomposes the transformation into smaller, reusable components, each with a precise security guarantee. This modular approach clarifies the trade-offs between assumptions, efficiency, tightness, and correctness, and has become the standard framework for the application of FO, particularly in the post-quantum setting.

The key insight of Hofheinz et al. is that the FO transformation can be understood as a

composition of two simpler transformations: T and U, each converting a weaker primitive into a stronger one.

Definition 3.14 Transformation T: From OW-CPA to OW-PCVA. Let PKE = (KeyGen, Encrypt, Decrypt) be a PKE scheme with message space $\mathcal{M}$ and coin space $\mathcal{R}$, and let $G : \mathcal{M} \to \mathcal{R}$ be a hash function. Define $\mathrm{PKE}_1 = \mathrm{T}[\mathrm{PKE}, G]$ as the deterministic encryption scheme with:

- $\mathrm{KeyGen}_1 = \mathrm{KeyGen}$.
- $\mathrm{Encrypt}_1(pk, m) = \mathrm{Encrypt}(pk, m; G(m))$.
- $\mathrm{Decrypt}_1(sk, c)$: Compute $m = \mathrm{Decrypt}(sk, c)$. If $m \neq \perp \wedge \mathrm{Encrypt}_1(pk, m) = c$, output m; otherwise output $\perp$.

The transformation T derandomizes encryption using G and enforces consistency via re-encryption at decryption time.

Definition 3.15 Transformation U: From OW-PCVA to IND-CCA. Given a deterministic encryption scheme PKE_1 (or more generally, a scheme with OW-PCVA security), and a hash function H, define $\mathrm{KEM} = \mathrm{U}[\mathrm{PKE}_1, H]$. There are four variants of U, differing in how the key K is derived and how invalid ciphertexts are rejected:

Table 3.1 Four variants of the transformation U.

Variant	Key Derivation	Rejection Type	Assumption Required
$\mathrm{U}^{\not\perp}$	$K = H(m, c)$	Implicit (output $H(c, s)$ for random s)	OW-PCA
$\mathrm{U}^{\perp}$	$K = H(m, c)$	Explicit (output $\perp$ on failure)	OW-PCVA
$\mathrm{U}^{\not\perp}_m$	$K = H(m)$	Implicit	det. OW-CPA
$\mathrm{U}^{\perp}_m$	$K = H(m)$	Explicit	det. OW-VA

All variants share the same encapsulation: using the sample random m and compute $c = \mathrm{Encrypt}_1(pk, m)$, output $(H(m, c), c)$ (or $(H(m), c)$ for the U_m variants). Decapsulation differs in how rejection is handled.

The full FO transformation is obtained by composing T and U, like:

$$\mathrm{FO}^{\not\perp}[\mathrm{PKE}, G, H] = \mathrm{U}^{\not\perp}[\mathrm{T}[\mathrm{PKE}, G], H].$$

3.3.4 Security Analysis

Theorem 3.16 Security of T. If PKE is δ-correct and OW-CPA secure, and G is a random oracle, then PKE_1 is OW-PCA secure and δ_1-correct. Specifically, for any OW-PCA adversary B against PKE_1 making at most q_G queries to G, there exists an OW-CPA adversary A against PKE such that:

$$\mathrm{Adv}^{\text{ow-pca}}_{\mathrm{PKE}_1}(B) \leq q_G \cdot \delta + (q_G + 1) \cdot \mathrm{Adv}^{\text{ow-cpa}}_{\mathrm{PKE}}(A).$$

with $\delta_1 = q_G \cdot \delta$-correctness.

Proof Sketch. The reduction works by simulating the plaintext checking oracle via re-encryption using the random oracle G, and leveraging the OW-CPA security of PKE to handle the challenge ciphertext. The correctness error δ accounts for the possibility that decryption may fail even for honestly generated ciphertexts—a crucial feature for lattice-based PKE schemes.

Theorem 3.17 Security of $\mathrm{U}^{\perp}$. If PKE_1 is OW-PCVA secure and δ_1-correct, and H is a random oracle, then $\mathrm{KEM} = \mathrm{U}^{\perp}[\mathrm{PKE}_1, H]$ is IND-CCA secure and δ_1-correct. Specifically, for any IND-CCA adversary B against KEM making at most q_D decapsulation queries and q_H queries to H, there exists an OW-PCVA adversary A against PKE_1 such that:

$$\mathrm{Adv}^{\text{ind-cca}}_{\mathrm{KEM}}(B) \leq \mathrm{Adv}^{\text{ow-pcva}}_{\mathrm{PKE}_1}(A).$$

Proof Sketch. The decapsulation oracle can be simulated without the secret key by using the plaintext checking oracle and ciphertext validity oracle provided by the OW-PCVA game. The simulation is perfect and straight-line, yielding a tight reduction.

The security proofs for $\mathrm{U}^{\not\perp}$, $\mathrm{U}^{\perp}_m$, and $\mathrm{U}^{\not\perp}_m$ follow similar patterns, with trade-offs between tightness and the strength of required assumptions. The $\mathrm{U}^{\not\perp}_m$ variant (implicit rejection, key derived from m only) is particularly important for practice, as it can be instantiated from the weaker OW-CPA assumption after applying T, and gracefully handles decryption errors.

Combined the security of T and U, we can get the security of KEM.

Theorem 3.18 Composition Theorem. If PKE is OW-CPA secure with δ-correctness, and G, H are random oracles, then $\mathrm{KEM}^{\not\perp} = \mathrm{FO}^{\not\perp}[\mathrm{PKE}, G, H]$ is IND-CCA secure. Specifically, for any IND-CCA adversary B against $\mathrm{KEM}^{\not\perp}$ making at most q_G queries to G and q_H queries to H, there exists an OW-CPA adversary A against PKE such that:

$$\mathrm{Adv}^{\text{ind-cca}}_{\mathrm{KEM}^{\not\perp}}(B) \leq 2q_{RO} \cdot \mathrm{Adv}^{\text{ow-cpa}}_{\mathrm{PKE}}(A) + \frac{2q_{RO}}{|\mathcal{M}|} + q_{RO} \cdot \delta,$$

where $q_{RO} = q_G + q_H$.

This bound is non-tight (loses a factor of q_{RO}), but the modular approach clarifies exactly where the loss occurs and how parameters should be chosen to maintain security.

3.3.5 The Quantum FO Transformation

With the advent of quantum computing, cryptographers must also consider adversaries equipped with quantum computers that can query random oracles in superposition. This scenario is captured by the **Quantum Random Oracle Model (QROM)**.

Targhi and Unruh[8] proved that a modified version of the FO transformation remains secure in the QROM. Their construction, which we denote $\mathrm{QFO}_m^{\perp}$, adds an additional hash value to the ciphertext:

Definition 3.19 Quantum-Safe FO, $\mathrm{QFO}_m^{\perp}$. Let PKE be an OW-CPA secure PKE, and let G, H, H' be hash functions. The QFO-transformed KEM is:

- **Encapsulation:** Sample random m, compute $r = G(m)$, $c = \mathrm{Encrypt}(pk, m; r)$, $d = H'(m)$, output $K = H(m)$ and ciphertext $\psi = (c, d)$.
- **Decapsulation:** Decrypt c to obtain m'. If $m' = \perp$ or $H'(m') \neq d$, output $\perp$; otherwise output $K = H(m')$.

The additional hash d serves as a "confirmation" that prevents certain quantum superposition attacks. Security in the QROM requires sub-exponential hardness assumptions (e.g., LWE with super-polynomial modulus-to-noise ratio) because the security reductions lose a factor exponential in the number of rewinding steps needed for extraction.

Theorem 3.20 QFO Security[7]. If PKE is OW-CPA secure with δ-correctness under sub-exponential hardness assumptions, then $\mathrm{QFO}_m^{\perp}[\mathrm{PKE}, G, H, H']$ is IND-CCA secure in the QROM. The concrete security bound is:

$$\mathrm{Adv}^{\text{ind-cca}}_{\mathrm{QFO}_m^{\perp}}(B) \leq 8q_{RO} \cdot \sqrt{\delta \cdot q_{RO}^2 + q_{RO} \cdot \sqrt{\mathrm{Adv}^{\text{ow-cpa}}_{\mathrm{PKE}}(A)}}.$$

where $q_{RO} = q_G + q_H + q_{H'}$.

The quadratic loss arises from the use of the one-way to hiding (OW2H) lemma, which is used to bound the advantage of an adversary that makes quantum superposition queries to the

random oracle. The OW2H lemma introduces a square-root factor because quantum adversaries can test multiple candidates simultaneously, requiring the reduction to "guess" which query was critical. This is analogous to the square-root speedup offered by Grover's algorithm, and the quadratic loss reflects the intrinsic difficulty of proving security against quantum adversaries generically.

3.4 Milestone 4: Kyber/ML-KEM and Post-Quantum Standardization

With the threat of quantum computers looming and the FO transformation providing a generic method to achieve CCA security, the cryptographic community has been actively standardizing post-quantum cryptographic algorithms. Among these, the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), formerly known as CRYSTALS-Kyber, has emerged as the primary KEM standard[4]. Its design exemplifies the principles we have developed throughout this paper: the KEM abstraction, FO transformation, and careful engineering of correctness and security.

3.4.1 The Underlying Module-LWE Problem

ML-KEM is based on the **Module Learning with Errors (Module-LWE)** problem, a generalization of Ring-LWE that offers a favorable trade-off between security and efficiency. Unlike Ring-LWE, which operates over a single polynomial ring and can be vulnerable to certain attacks when the ring has special structure, Module-LWE operates over a module of small rank k (e.g., $k = 2, 3, 4$), balancing security and performance.

Definition 3.21 Module-LWE. Let $R_q = \mathbb{Z}_q[X]/(X^n + 1)$ be a cyclotomic ring with $n = 256$, R_q^k the set of k-dimensional vectors over R_q, and χ a distribution over R_q producing small-norm elements (typically centered binomial). For a secret vector $\boldsymbol{s} \in R_q^k$, the Module-LWE distribution $\mathcal{A}_{\boldsymbol{s},\chi}$ outputs samples $(\boldsymbol{a}, \boldsymbol{a}^\top \boldsymbol{s} + e)$ where $\boldsymbol{a} \leftarrow R_q^k$ uniformly and $e \leftarrow \chi$. The search Module-LWE problem asks to recover $\boldsymbol{s}$ from such samples; the decision version asks to distinguish $\mathcal{A}_{\boldsymbol{s},\chi}$ from uniform.

ML-KEM uses parameters $n = 256$, $k \in \{2, 3, 4\}$ (security levels 1, 3, 5 respectively), and modulus $q = 3329$. These parameters are chosen to balance security (against both clas-

sical and quantum attacks) with performance while minimizing decryption error probability to approximately 2^{-140}.

Why Module-LWE? The module structure offers a "sweet spot" in the lattice hierarchy. Standard LWE with large dimension is secure but inefficient; Ring-LWE is efficient but its algebraic structure may enable certain attacks (e.g., exploiting the ring's automorphisms). Module-LWE with small rank k (e.g., $k = 2, 3, 4$) inherits the efficiency of Ring-LWE while diluting the algebraic structure, providing a conservative security margin. This parameterization allows ML-KEM to achieve a compact public key size (e.g., 800 bytes for $k = 2$) and fast operations while maintaining strong security guarantees.

3.4.2 The CPA-Secure Base PKE: Primal Regev

The Kyber CPA-secure PKE (the base scheme) is a standard Module-LWE encryption scheme. Let χ be a centered binomial distribution (specifically, η_1, η_2 parameters for different noise levels). The scheme proceeds as follows:

- **KeyGen:** Sample $\boldsymbol{A} \leftarrow R_q^{k \times k}$ uniformly, $\boldsymbol{s} \leftarrow \chi^k$, $\boldsymbol{e} \leftarrow \chi^k$, compute $\boldsymbol{t} = \boldsymbol{A}\boldsymbol{s} + \boldsymbol{e}$. Output $pk = (\boldsymbol{A}, \boldsymbol{t})$, $sk = \boldsymbol{s}$.
- **Encrypt:** Sample $\boldsymbol{r} \leftarrow \chi^k$, $\boldsymbol{e}_1 \leftarrow \chi^k$, $e_2 \leftarrow \chi$, compute $\boldsymbol{u} = \boldsymbol{A}^\top \boldsymbol{r} + \boldsymbol{e}_1$, $v = \boldsymbol{t}^\top \boldsymbol{r} + e_2 + \lfloor q/2 \rceil \cdot m$. Output $c = (\boldsymbol{u}, v)$.
- **Decrypt:** Compute $w = v - \boldsymbol{s}^\top \boldsymbol{u}$. If w is closer to $\lfloor q/2 \rceil$ than to 0 modulo q, output 1; else output 0.

Correctness Analysis. For an honestly generated ciphertext, we have:

$$\begin{aligned} w &= v - \boldsymbol{s}^\top \boldsymbol{u} \\ &= (\boldsymbol{t}^\top \boldsymbol{r} + e_2 + \lfloor q/2 \rceil \cdot m) - \boldsymbol{s}^\top (\boldsymbol{A}^\top \boldsymbol{r} + \boldsymbol{e}_1) \\ &= (\boldsymbol{t}^\top \boldsymbol{r} - \boldsymbol{s}^\top \boldsymbol{A}^\top \boldsymbol{r}) + (e_2 - \boldsymbol{s}^\top \boldsymbol{e}_1) + \lfloor q/2 \rceil \cdot m. \end{aligned}$$

Since $\boldsymbol{t} = \boldsymbol{A}\boldsymbol{s} + \boldsymbol{e}$, we have $\boldsymbol{t}^\top = \boldsymbol{s}^\top \boldsymbol{A}^\top + \boldsymbol{e}^\top$. Substituting:

$$\boldsymbol{t}^\top \boldsymbol{r} - \boldsymbol{s}^\top \boldsymbol{A}^\top \boldsymbol{r} = (\boldsymbol{s}^\top \boldsymbol{A}^\top + \boldsymbol{e}^\top)\boldsymbol{r} - \boldsymbol{s}^\top \boldsymbol{A}^\top \boldsymbol{r} = \boldsymbol{e}^\top \boldsymbol{r}.$$

Therefore:

$$w = \boldsymbol{e}^\top \boldsymbol{r} + e_2 - \boldsymbol{s}^\top \boldsymbol{e}_1 + \lfloor q/2 \rceil \cdot m.$$

All noise terms $\boldsymbol{e}^\top \boldsymbol{r}$, e_2, and $\boldsymbol{s}^\top \boldsymbol{e}_1$ are small (bounded by some $B \ll q/4$ with overwhelming probability). Thus w is close to $\lfloor q/2 \rceil \cdot m$. The decryption correctly outputs m when $|w - \lfloor q/2 \rceil \cdot m| < q/4$, which holds except with probability $\delta \approx 2^{-140}$ for the chosen parameters.

3.4.3 The FO Transformation Applied to Kyber

This base scheme achieves only IND-CPA security. To obtain IND-CCA security, ML-KEM applies the $\mathrm{FO}_m^{\not\perp}$ transformation (implicit rejection, key from m only), which we denote as **Kyber.CCAKEM**:

- **Key Generation:** Same as Kyber.CPA.KeyGen.
- **Encapsulation:** On input pk:
  1. Sample a random message $m \leftarrow \{0,1\}^{256}$.
  2. Compute $(\boldsymbol{u}, v) = \text{Kyber.CPA.Encrypt}(pk, m; G(m))$, where G is a hash function (SHAKE-256 in practice, with domain separation).
  3. Compute shared secret $K = H(m)$.
  4. Output $(K, (\boldsymbol{u}, v))$.
- **Decapsulation:** On input sk and the ciphertext $c = (\boldsymbol{u}, v)$:
  1. Decrypt to obtain $m' = \text{Kyber.CPA.Decrypt}(sk, (\boldsymbol{u}, v))$.
  2. If $m' = \perp$, output $\perp$.
  3. Compute $(\boldsymbol{u}', v') = \text{Kyber.CPA.Encrypt}(pk, m'; G(m'))$.
  4. If $(\boldsymbol{u}', v') = (\boldsymbol{u}, v)$, output $K = H(m')$; otherwise output $\perp$.

The actual ML-KEM specification (FIPS 203) uses a slightly different design with **key derivation functions** for domain separation and includes an additional "key confirmation" hash to prevent certain fault attacks. It also employs a **key pair validation** step during decapsulation to reject malformed ciphertexts.

3.4.4 Security Analysis

The security of ML-KEM follows from the hardness of Module-LWE and the security of the FO transformation in the ROM. By the modular analysis of Hofheinz et al.[7], we have:

$$\mathrm{Adv}^{\text{ind-cca}}_{\text{ML-KEM}}(B) \le 3 \cdot \mathrm{Adv}^{\text{mlwe}}_{\text{MLWE}}(A) + \frac{3q_G + 3q_H}{2^{256}} + (2q_G + 2q_H + q_D) \cdot \delta,$$

where $\delta \approx 2^{-140}$ is the decryption error probability, q_D, q_G, q_H are numbers of decapsulation queries and hash queries (bounded by, say, 2^{64} in practice), and A is an adversary against Module-LWE with comparable running time. The term $(2q_G + 2q_H + q_D) \cdot \delta$ remains negligible (e.g., $5 \cdot 2^{64} \cdot 2^{-140} < 2^{-73}$), so the dominant term is the Module-LWE advantage.

The NIST security analysis concluded that with appropriate parameter choices (e.g., $k = 3$, $n = 256$, $q = 3329$ for security level 3), ML-KEM provides 128 bits of security against both classical and quantum adversaries. The IND-CCA security bound ensures that even active attackers who can inject chosen ciphertexts cannot recover the shared secret or distinguish it from random.

3.4.5 Efficiency Trade-offs and Parameter Selection

ML-KEM offers three security levels corresponding to $k = 2, 3, 4$, showing in Table 3.2:

Table 3.2 ML-KEM standard security levels and corresponding parameter sizes

Level	k	Public Key Size	Ciphertext Size	Security (bits)
1	2	800 bytes	768 bytes	128
3	3	1184 bytes	1088 bytes	192
5	4	1568 bytes	1568 bytes	256

The choice of $q = 3329$ (a prime of the form $3329 = 256 \cdot 13 + 1$) enables efficient Number Theoretic Transform (NTT) operations, which reduce multiplication complexity from $O(n^2)$ to $O(n \log n)$. The polynomial degree $n = 256$ ensures that NTT operations fit within modern CPU cache hierarchies, enabling fast constant-time implementations.

3.4.6 Comparison with Other NIST KEM Finalists

While ML-KEM was selected as the primary KEM standard, NIST also considered other candidates that illustrate different design philosophies:

- **Classic McEliece**[9]: A code-based KEM with 50+ years of cryptanalysis. It offers extremely fast encryption and decryption but produces very large public keys (hundreds of

kilobytes to megabytes). Unlike ML-KEM, Classic McEliece does not use the FO transformation in the standard way; instead, it uses a variant of the Niederreiter framework with a different approach to achieving CCA security.

- **BIKE**[10]: A code-based KEM using QC-MDPC codes. It offers smaller key sizes than Classic McEliece but has a non-zero decryption failure rate that requires careful handling in the FO transformation. The security analysis of BIKE is less mature than that of ML-KEM, and its implementation requires constant-time decoding algorithms to avoid side-channel attacks.
- **HQC**[11]: Another code-based KEM with a provably low decryption error rate. It provides strong security guarantees but is less efficient than ML-KEM.

The selection of ML-KEM as the primary standard reflects a judgment that lattice-based cryptography offers the best balance of security, performance, and implementation simplicity for general-purpose use.

Chapter 4 Implementation and Evaluation

A Rust Implementation of ML-KEM

The preceding chapters introduced the KEM abstraction and traced its technical development from KEM/DEM composition to ML-KEM. This chapter presents the implementation component of the project. The implementation, named `jkem`, is a teaching-oriented Rust implementation of ML-KEM following FIPS 203[4]. It supports the standard ML-KEM-512, ML-KEM-768, and ML-KEM-1024 parameter sets, and it is used in this report to study the concrete engineering interface between the algebraic K-PKE core, the Fujisaki-Okamoto style transform, and side-channel-aware implementation techniques. The source code is available at https://github.com/wsm25/jkem, and the generated documentation is available at https://wsm.moe/jkem/jkem/.

4.1 Implementation Design

4.1.1 Implementation Goal and Scope

The implementation has three main objectives. First, it aims to conform to the standard behavior of ML-KEM, as verified by deterministic Known Answer Tests (KATs) for keys, ciphertexts, and shared secrets. Second, it separates polynomial arithmetic, sampling, K-PKE algorithms, and ML-KEM control logic into distinct modules, making the correspondence between the specification and the code explicit. Third, it provides sufficient performance for empirical evaluation against an optimized baseline, while retaining a portable Rust implementation without architecture-specific assembly optimizations.

The public API exposes a generic type `MlKem<P>` parameterized by an ML-KEM parameter set, together with convenience aliases `MlKem512`, `MlKem768`, and `MlKem1024`. The three public operations follow the KEM syntax:

$$\texttt{keygen()} \to (\texttt{ek}, \texttt{dk}), \qquad \texttt{encaps(ek)} \to (\texttt{ct}, K), \qquad \texttt{decaps(dk, ct)} \to K.$$

Here `ek` is the encapsulation key, `dk` is the decapsulation key, `ct` is the ciphertext, and K is the 32-byte shared secret.

4.1.2 Dependencies and References

The implementation uses a limited dependency set. The `getrandom` crate supplies operating-system randomness for public key-generation and encapsulation APIs. The `sha3` crate supplies SHA3 and SHAKE functions required by ML-KEM, including SHA3-256, SHA3-512, SHAKE128, and SHAKE256. The `subtle` crate is used for constant-time equality checks and conditional selection in decapsulation. The `zeroize` crate wipes selected temporary secret byte buffers. The `hybrid-array` crate provides fixed-size arrays indexed by type-level parameter sizes, which makes the same generic implementation cover all standard ML-KEM parameter sets.

The primary specification reference is FIPS 203[4]. We also use the Kyber/ML-KEM design lineage[12] to interpret the relationship between the module-lattice PKE core and the final CCA-secure KEM interface. The implementation is structured around the algorithms and byte layouts specified by the standard, rather than around the organization of any single reference implementation. Deterministic KAT files used for validation were obtained from Github Gist.

4.1.3 Architecture

The implementation hierarchy follows the organization of FIPS 203[4]. The standard first specifies auxiliary algorithms, then the K-PKE component scheme, then the deterministic internal ML-KEM algorithms, and finally the public ML-KEM interface. The codebase preserves this ordering: each layer depends on the layers below it and exposes only the interface needed by the next layer.

1. **Parameter-set layer.** Corresponding to FIPS 203 Section 8, the `params` module defines $n, q, k, \eta_1, \eta_2, d_u, d_v$ and the derived byte lengths for each approved parameter set. These definitions determine the dimensions of vectors, matrices, keys, and ciphertexts at compile time.
2. **Auxiliary-algorithm layer.** Corresponding to FIPS 203 Section 4, the `math`, `security`, and serialization code implement the algorithms that are reused by the

higher-level procedures. This layer includes arithmetic in

$$R_q = \mathbb{Z}_q[X]/(X^{256} + 1), \qquad q = 3329,$$

polynomial and vector operations, the NTT and inverse NTT, byte encoding/decoding, compression/decompression, SHA3/SHAKE-based functions, matrix sampling, and centered binomial sampling.

3. **K-PKE component layer.** Corresponding to FIPS 203 Section 5, the `kpke` module implements `K-PKE.KeyGen`, `K-PKE.Encrypt`, and `K-PKE.Decrypt`. This layer is the algebraic public-key encryption component used internally by ML-KEM.
4. **Internal ML-KEM layer.** Corresponding to FIPS 203 Section 6, the `control` module and the internal methods `keygen_internal`, `encaps_internal`, and the deterministic core of `decaps` implement `ML-KEM.KeyGen_internal`, `ML-KEM.Encaps_internal`, and `ML-KEM.Decaps_internal`. This layer performs key packing, public-key hash binding, derivation of shared secrets and encryption coins, re-encryption validation, and implicit rejection.
5. **Public ML-KEM layer.** Corresponding to FIPS 203 Section 7, the generic type `MlKem<P>` exposes `keygen`, `encaps`, and `decaps`. This layer obtains randomness for public key generation and encapsulation, and invokes the deterministic internal layer.

This hierarchy is important because FIPS 203 distinguishes deterministic internal algorithms from the randomized public KEM interface. In particular, `ML-KEM.KeyGen_internal`, `ML-KEM.Encaps_internal`, and `ML-KEM.Decaps_internal` are deterministic once their inputs are fixed, whereas the public `ML-KEM.KeyGen` and `ML-KEM.Encaps` algorithms generate fresh randomness before calling the internal algorithms. The same separation is reflected in the implementation and is also used by the KAT tests.

4.2 Implementation Details

4.2.1 Implementation of the ML-KEM Algorithms

The theoretical role of KEM/DEM composition has been discussed in Chapter 3. This section therefore focuses on the FIPS 203 algorithm hierarchy as realized by the implementation.

Key generation. The public `keygen` API implements `ML-KEM.KeyGen`: it draws two 32-byte random values d and z, then calls the deterministic internal algorithm. The internal

algorithm first invokes `K-PKE.KeyGen`. In FIPS 203, this K-PKE procedure expands d into ρ, used for public matrix sampling, and σ, used for secret noise sampling. In the implementation, this expansion is factored into the control layer before calling the K-PKE arithmetic routine, but the resulting ρ, σ data flow is the same. The K-PKE component samples the public matrix in the NTT domain, samples the secret and error vectors, and computes the public-key polynomial vector. The ML-KEM internal layer then sets `ek` = `ekPKE` and packs the decapsulation key as

$$\texttt{dk} = \texttt{dkPKE} \parallel \texttt{ek} \parallel H(\texttt{ek}) \parallel z,$$

where z is the secret fallback value used for implicit rejection.

Encapsulation. The public `encaps` API implements `ML-KEM.Encaps`: it draws a fresh 32-byte value m, then calls `ML-KEM.Encaps_internal`. The internal algorithm computes

$$(K, r) = G(m \parallel H(\texttt{ek})),$$

where K is the shared secret and r is the deterministic encryption coins. It then invokes `K-PKE.Encrypt` with `ek`, m, and r. The K-PKE component regenerates the public matrix from ρ, samples the encryption noise from r, computes the ciphertext components, compresses and serializes them, and returns the ciphertext. The public API returns this ciphertext together with K.

Decapsulation. The public `decaps` API implements `ML-KEM.Decaps`. Since FIPS 203 specifies decapsulation as deterministic, this API does not draw randomness. It parses the ML-KEM decapsulation key into `dkPKE`, `ek`, $h = H(\texttt{ek})$, and z. The K-PKE component decrypts the ciphertext to recover a candidate message m'. The internal ML-KEM layer derives $(K', r') = G(m' \parallel h)$, computes the fallback secret $J(z \parallel \texttt{ct})$, and invokes `K-PKE.Encrypt` to re-encrypt m' using r'. The re-encrypted ciphertext is compared with the received ciphertext to determine whether decapsulation follows the success path or the implicit-rejection path.

The final selection implements the Fujisaki-Okamoto style implicit rejection step. For a valid ciphertext, decapsulation returns the derived success secret. For an invalid ciphertext, it returns

$$J(z \parallel \texttt{ct}),$$

implemented with SHAKE256. This design avoids exposing ciphertext validity through an explicit failure return. In the implementation, the success secret and fallback secret are selected byte-by-byte using constant-time conditional selection from `subtle`.

4.2.2 Parameter Abstraction

The standard parameter sets are represented by marker types implementing a common parameter trait. This trait fixes the module rank k, noise parameters η_1, η_2, compression parameters d_u, d_v, and the resulting byte lengths for keys and ciphertexts. The three standard instances are summarized in Table 4.1.

Table 4.1 ML-KEM standard parameter sets.

Parameter set	k	η_1	η_2	d_u	d_v
ML-KEM-512	2	3	2	10	4
ML-KEM-768	3	2	2	10	4
ML-KEM-1024	4	2	2	11	5

Since the algorithms are generic over this trait, most implementation code is shared across all three parameter sets. The benchmark code additionally defines a non-standard experimental $k = 5$ instance to evaluate whether the structure extends beyond the standardized choices. This instance is used only as an engineering extensibility experiment and is not a proposed cryptographic parameter set.

4.2.3 Security Engineering Considerations

The security engineering layer is intended to preserve the security semantics of ML-KEM in the executable implementation. Randomness is generated only at the public API boundary: key generation obtains d and z, and encapsulation obtains m, from the operating-system random source. Deterministic internal hooks are therefore reserved for KAT reproduction and timing measurements. During decapsulation, the implementation computes both the candidate shared secret and the implicit-rejection fallback secret, compares ciphertexts with constant-time equality, and uses constant-time conditional selection for the returned secret. Selected secret byte buffers are also zeroed after use.

These measures should be understood as engineering precautions rather than as a formal constant-time proof or production security audit.

4.3 Evaluation

4.3.1 Testing

Correctness is tested against deterministic KAT files for ML-KEM-512, ML-KEM-768, and ML-KEM-1024. For each parameter set, the tests verify that the KAT file is present and unchanged, then check generated public keys, generated decapsulation keys, generated ciphertexts, encapsulated shared secrets, and decapsulated shared secrets. Each file contributes 100 test cases. The tests are included in the public codebase, so the reported results can be reproduced by running `cargo test` from the project root.

Table 4.2 KAT integration test matrix.

Parameter set	File	PK	SK	CT	Enc SS	Dec SS	Malformed
ML-KEM-512	✓	✓	✓	✓	✓	✓	✓
ML-KEM-768	✓	✓	✓	✓	✓	✓	✓
ML-KEM-1024	✓	✓	✓	✓	✓	✓	✓

The tests also cover malformed-input behavior. Modified ciphertexts do not produce explicit validity failures; instead, decapsulation returns the implicit rejection secret $J(\texttt{z} \parallel \texttt{ct})$. Malformed encapsulation keys with invalid encoded coefficients are rejected, and decapsulation keys with an incorrect stored $H(\texttt{ek})$ are rejected. The integration test suite contains 21 tests across the three parameter sets.

4.3.2 Benchmark Methodology

The benchmark evaluates the three public KEM operations: key generation, encapsulation, and decapsulation. Each operation is measured repeatedly and reported as time per operation, together with a normalized throughput estimate. The comparison baseline is `mlkem-native`, an optimized implementation with native low-level routines. This comparison highlights the performance cost of the portable generic Rust implementation relative to an implementation engineered primarily for high throughput.

The throughput measurements in Table 4.3 show a clear performance gap. Across the three standard parameter sets, the Rust implementation is slower than the optimized native baseline

Table 4.3 Mean benchmark throughput in operations per second.

Case	Key generation	Encapsulation	Decapsulation
native/mlkem512	40984	34400	27078
native/mlkem768	25967	21820	17803
native/mlkem1024	16978	14986	12541
jkem/mlkem512	9867	7867	5371
jkem/mlkem768	5844	4980	3531
jkem/mlkem1024	3799	3365	2485

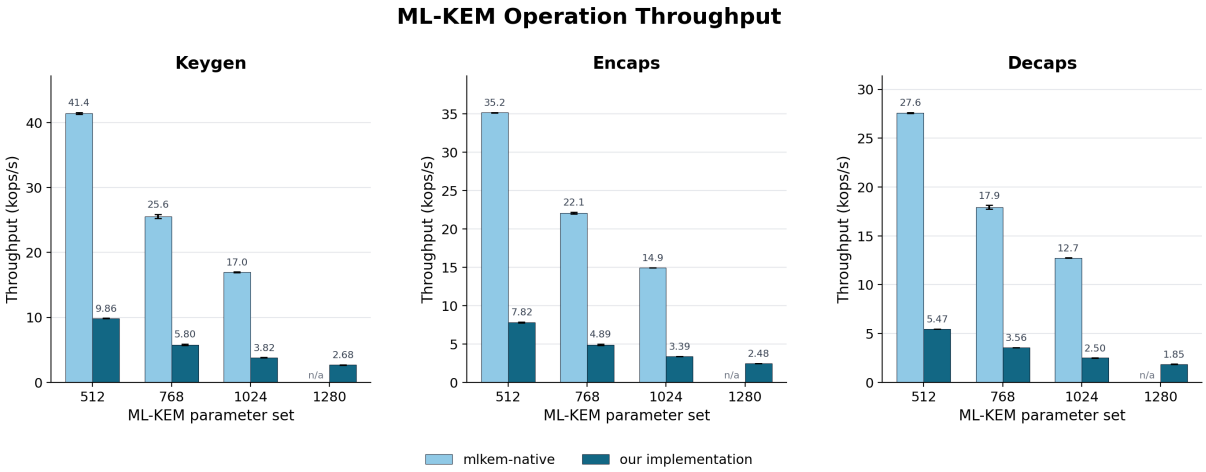

Figure 4.1 Operation throughput for key generation, encapsulation, and decapsulation.

while preserving a simpler and more inspectable structure.

The benchmark suite also includes two traditional key-exchange baselines using OpenSSL: X25519 and finite-field Diffie-Hellman with the `modp2048-256` group. These benchmarks are not operation-for-operation equivalent to a KEM, because DH-style key exchange is symmetric while a KEM has separate encapsulation and decapsulation roles. A useful comparison is therefore the local work done by each endpoint in one unauthenticated key-establishment run. For DH, each endpoint performs one ephemeral key generation and one shared secret derivation. For ML-KEM-512, the encapsulating endpoint performs encapsulation, while the decapsulating endpoint performs key generation and decapsulation.

Table 4.4 shows that the portable Rust ML-KEM-512 implementation is slower than OpenSSL X25519 when both endpoints' local work is added together: about 414.65 μs versus 217.75 μs, or roughly 1.90× the CPU time. The split is asymmetric: the encapsulating side

Table 4.4 Endpoint CPU time for one key-establishment run.

Case	Endpoint A	Endpoint B	Total local CPU time
`jkem/mlkem512`	287.53 μs	127.11 μs	414.65 μs
`native/mlkem512`	61.33 μs	29.07 μs	90.40 μs
`openssl/x25519`	108.87 μs	108.87 μs	217.75 μs
`openssl/modp2048-256`	1236.71 μs	1236.71 μs	2473.42 μs

alone is close to one X25519 endpoint, while the key-generation and decapsulation side is more expensive. Against traditional finite-field Diffie-Hellman at the `modp2048-256` group, even this portable ML-KEM-512 implementation is substantially faster; its total local CPU time is about 0.17× that of the two DH endpoints. The optimized `mlkem-native` ML-KEM-512 baseline is faster than both DH baselines in this measurement.

To estimate operation bandwidth, public key and ciphertext byte lengths are not a stable measure of the internal work done by ML-KEM, because the dominant algebraic work scales with the module rank k. To relate the measured operation throughput to k, we derive an approximate internal-data throughput. Let $N = 256$ be the number of coefficients in one polynomial and let $s_c = 2$ bytes be the in-memory size of one coefficient. One polynomial therefore accounts for

$$B_{\text{poly}} = Ns_c = 512 \text{ bytes}.$$

The normalization estimates the amount of polynomial data touched by the dominant algebraic operations. The k^2 term represents matrix generation and matrix-vector multiplication. The linear terms represent vector NTTs, inverse NTTs, noise polynomials, encoding/decoding, and additional dot products. The constant term accounts for the single-polynomial message or ciphertext component. Thus,

$$\begin{aligned} D_{\text{keygen}}(k) &= B_{\text{poly}}(k^2 + 4k), \\ D_{\text{encaps}}(k) &= B_{\text{poly}}(k^2 + 5k + 1), \\ D_{\text{decaps}}(k) &= B_{\text{poly}}(k^2 + 6k + 1). \end{aligned}$$

For an operation with measured time $t_{\text{op}}(k)$ seconds, the normalized throughput is

$$\text{MB/s}_{\text{real}}(\text{op}, k) = \frac{D_{\text{op}}(k)}{10^6 t_{\text{op}}(k)}.$$

Equivalently, if the benchmark reports $Q_{\mathrm{op}}(k)$ operations per second, then

$$\mathrm{MB/s}_{\mathrm{real}}(\mathrm{op}, k) = \frac{D_{\mathrm{op}}(k) Q_{\mathrm{op}}(k)}{10^6}.$$

This model is a first-order approximation: it does not separately weight fixed overhead, rejection sampling, SHA3/SHAKE setup, compression widths, or decapsulation's validation re-encryption. It is nevertheless sufficient to compare how throughput scales with k.

Table 4.5 Normalized internal-data throughput derived from Table 4.3.

Case	Key generation	Encapsulation	Decapsulation
`native/mlkem512`	251.81	264.19	235.69
`native/mlkem768`	279.20	279.30	255.22
`native/mlkem1024`	278.17	283.89	263.26
`jkem/mlkem512`	60.62	60.42	46.75
`jkem/mlkem768`	62.83	63.74	50.62
`jkem/mlkem1024`	62.24	63.75	52.17

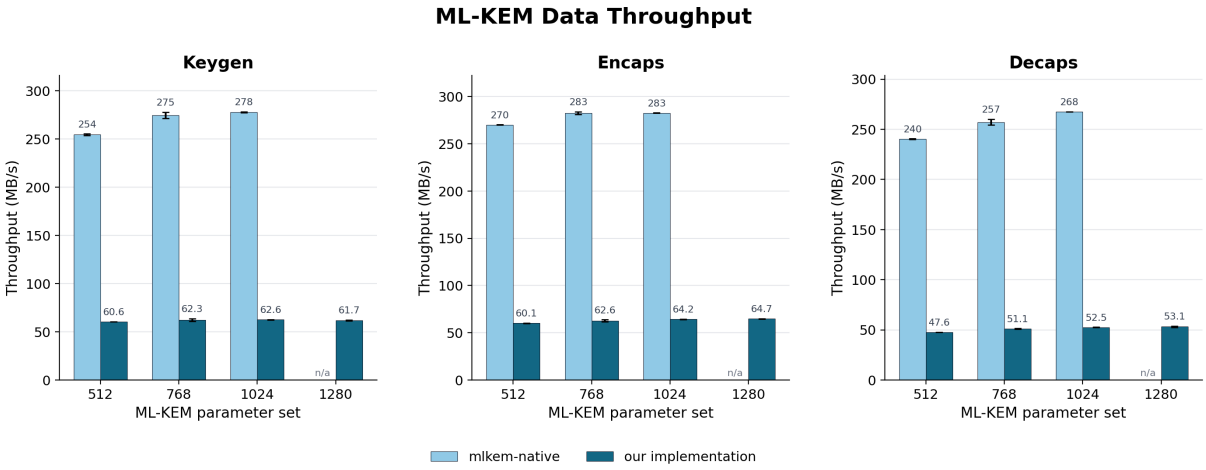

Figure 4.2 Approximate normalized internal-data throughput.

Table 4.5 verifies the values plotted in Figure 4.2: the graph is obtained by multiplying each operations-per-second value in Table 4.3 by the corresponding $D_{\mathrm{op}}(k)$ and dividing by 10^6.

4.3.3 Constant Timing Evaluation

Welch t-tests in the style of dudect are used to search for timing distinguishers. For encapsulation, the deterministic internal hook allows the message to be classified as fixed or random

without measuring operating-system randomness. For decapsulation, the two classes are valid ciphertexts and one-bit-modified invalid ciphertexts; this directly exercises the implicit rejection path. A separate key-generation scan fixes the main seed and varies the secret fallback value z.

Table 4.6 Constant-timing test summary.

Test	Samples	Maximum t-statistic
Key generation z-scan	1024×128 cases	2.31
Encapsulation	0.069M samples	1.92
Decapsulation	0.100M samples	1.62

The observed results did not indicate a strong timing distinguisher in these runs. As summarized in Table 4.6, all reported t-statistics remain below the conventional dudect warning threshold. These numbers provide empirical evidence that the measured paths do not exhibit a statistically significant timing difference under this test setup. They do not provide a formal constant-time guarantee.

4.4 Discussion

The implementation confirms the architectural point developed in the earlier chapters: ML-KEM is best understood as a layered construction. The module-lattice K-PKE core supplies a noisy but correct CPA-secure encryption mechanism; the control plane binds public keys, derives coins and shared secrets, and enforces implicit rejection; the public KEM API exposes only key generation, encapsulation, and decapsulation. This separation makes the design easier to test and reason about, and it matches the KEM abstraction used in modern protocols.

The main limitation of the implementation is performance. Without architecture-specific vectorization or assembly, polynomial multiplication, NTT operations, sampling, and hashing remain significantly slower than optimized native implementations. Even with this limitation, the implementation is sufficient for experimental evaluation and for demonstrating how the formal KEM construction is realized as executable software.

Chapter 5 Applications and Open Problems

KEM in Protocol Deployment: HPKE and Hybrid TLS 1.3

The previous chapters developed KEM from its formal syntax, generic transformations, and concrete ML-KEM implementation. This chapter considers its role in deployed protocols. A KEM establishes a shared secret; the protocol around it must still provide context binding, authentication, downgrade resistance, traffic encryption, and careful implementation.

5.1 HPKE: A Clean Standardized KEM Application

Hybrid Public Key Encryption (HPKE), standardized as RFC 9180, is one of the clearest applications of the KEM abstraction[3]. Its architecture is modular:

- **KEM:** Establish a shared secret between sender and receiver.
- **KDF:** Bind the shared secret to protocol context and derive independent keys.
- **AEAD:** Encrypt and authenticate the actual application payload.

HPKE supports four modes: **base**, **PSK**, **auth**, and **authPSK**. These modes show what KEM deliberately does *not* provide. A KEM gives secrecy for the encapsulated shared secret; sender authentication, PSK binding, and application context binding are added by the surrounding HPKE mode and key schedule.

The security of HPKE comes from composition. An IND-CCA secure KEM protects the encapsulated secret; the KDF provides domain separation and context binding; the AEAD provides payload confidentiality and ciphertext integrity. Removing any layer changes the protocol's security meaning.

HPKE also illustrates a subtle point about forward secrecy. Erasing the sender's ephemeral randomness protects against later sender-side compromise, but compromise of the receiver's long-term decapsulation key can still expose past recorded encapsulations. HPKE is therefore not automatically equivalent to an authenticated ephemeral Diffie–Hellman handshake.

5.2 Hybrid Post-Quantum Key Exchange in TLS 1.3

TLS 1.3 separates key exchange, authentication, transcript hashing, key derivation, and record protection. It uses ephemeral DH/ECDHE shares for forward secrecy[13]. During post-quantum migration, replacing ECDHE outright with a KEM would abandon the classical assumption before the post-quantum ecosystem is fully mature.

The common migration strategy is hybrid key exchange: run a classical ephemeral exchange and a post-quantum KEM in the same handshake, then combine both outputs before entering the TLS key schedule. The IETF hybrid design draft describes this as a concatenation-based construction negotiated as a single TLS key exchange method[14]. The ECDHE-MLKEM draft gives concrete combinations such as `X25519MLKEM768`, `SecP256r1MLKEM768`, and `SecP384r1MLKEM1024`[15].

At a high level, the client sends a classical ECDHE key share and an ML-KEM encapsulation key. The server returns its ECDHE share and an ML-KEM ciphertext. The combined secret enters the TLS 1.3 key schedule. The goal is robustness: the handshake should retain a margin if either the classical or post-quantum assumption remains sound.

The transcript hash is central. TLS 1.3 authenticates values derived from the transcript, including algorithm negotiation, key-share messages, certificates, and other handshake data[13]. In a hybrid handshake, this binding must cover the selected hybrid group and both key-exchange components; otherwise, an attacker may try to remove the post-quantum component or force a weaker configuration.

Hybrid deployment also creates engineering costs. ML-KEM public keys and ciphertexts are much larger than X25519 or P-256 key shares, so `ClientHello` and `ServerHello` grow. Larger first flights can trigger fragmentation, middlebox failures, and latency. Implementations must also validate encoded public keys and ciphertexts without side-channel leakage.

Thus, hybrid TLS shows that post-quantum KEM deployment is not a simple substitution of ML-KEM for ECDH. It changes negotiation, wire format, transcript binding, implementation paths, and the security proof surface.

5.3 Open Problems

Current approaches still have several limitations. TLS 1.3 analysis already shows that real handshake security depends on authentication levels, transcript binding, key schedule separation, replay behavior, and forward secrecy, not only on one key-exchange primitive[16]. Adding a post-quantum KEM enlarges this proof surface. Hybrid constructions also need precise arguments for the intended "secure if one component survives" property[17]. At the same time, experiments on TLS and SSH show that post-quantum integration affects latency, packet sizing, TCP behavior, and server throughput[18].

The resulting research directions are closely connected. Protocol analysis needs tighter composable models for hybrid KEM use inside full handshakes, including the combiner, negotiation transcript, downgrade behavior, and decapsulation failure handling. Systems work needs lower-cost post-quantum handshakes through smaller wire formats, better batching, and cleaner TLS-library integration; existing implementation work shows that software-layer choices matter substantially[19]. Finally, malformed KEM inputs, fallback paths, and failure handling must remain compatible with constant-time handling. The post-quantum transition is therefore not only algorithm standardization, but also protocol design and systems security.

Chapter 6 Reflections and Insights

KEM is best understood as a disciplined interface for key establishment. It does not solve the whole secure-channel problem, but it gives protocols a clean way to obtain a high-entropy shared secret and then delegate the remaining work to KDF, AEAD, and authentication layers.

6.1 Strengths of the KEM Abstraction

The main advantage of KEM is modularity. The public-key algorithm is used only to establish a short shared secret; bulk data protection is left to symmetric cryptography. This matches the KEM/DEM view of hybrid encryption, where an IND-CCA secure KEM can be composed with a secure DEM to obtain a secure hybrid scheme[2]. It also matches deployed designs such as HPKE, where KEM, KDF, and AEAD have clearly separated roles[3].

KEM is also a natural fit for post-quantum cryptography. Lattice-based schemes such as ML-KEM are built from noisy algebraic primitives, and it is simpler to derive a random shared key than to provide a general arbitrary-message PKE interface. The Fujisaki-Okamoto transformation explains how a CPA-secure core can be lifted to a CCA-secure KEM[6-7]; ML-KEM standardizes this style of construction for module lattices[4,12].

Finally, the interface is implementation-friendly. The KEM operation surface is small, and in ML-KEM the expensive operations are regular: polynomial arithmetic, sampling, and SHA3/SHAKE. This makes optimized software and future hardware acceleration plausible.

6.2 Limitations

KEM does not remove the cost of the underlying assumption. ML-KEM public keys and ciphertexts are much larger than classical ECDH key shares, which affects packet size, fragmentation, latency, and compatibility. Measurements on post-quantum TLS and SSH show that deployment cost is not only CPU time, but also network behavior and server throughput[18].

The engineering ecosystem is also less mature than DH/ECDH. Classical key exchange has decades of implementation and protocol experience, while post-quantum KEM deployment is

still converging around validation, constant-time decapsulation, hybrid negotiation, and library APIs. A KEM alone is not an authenticated key exchange; authentication, downgrade protection, replay behavior, and transcript binding must come from the surrounding protocol, as in TLS 1.3[13,16].

Security also depends on assumptions and implementation discipline. ML-KEM relies on module-lattice hardness, and FO-style decapsulation must avoid leaking validity information through timing or failure behavior. Implicit rejection helps, but it does not replace careful constant-time implementation.

6.3 Research Directions

The first practical direction is dedicated acceleration. ML-KEM would benefit from instruction-set or hardware support for NTT, modular arithmetic over $q = 3329$, SHA3/SHAKE, and sampling. As with AES-NI, the value is both speed and a smaller side-channel surface.

The second direction is better transformations. FO-like transforms remain the bridge from weak public-key primitives to CCA-secure KEMs. More efficient or tighter variants could reduce parameter sizes, improve security margins, or handle imperfect correctness more cleanly.

The third direction is proof with tighter QROM security. Existing FO-style analyses in the Quantum Random Oracle Model often lose tightness[7-8]. Sharper reductions would make concrete post-quantum parameter choices easier to justify.

The final direction is protocol-realistic hybrid KEM analysis. Real deployments combine classical ECDHE and post-quantum KEMs inside full handshakes, with algorithm negotiation, transcript hashes, error handling, and downgrade resistance. Security models should capture the intended hybrid guarantee: the exchange should remain secure if at least one component remains secure[17].

6.4 Personal Reflections

Having worked through the technical development from Shoup's KEM/DEM systematization, the Cramer-Shoup KEM, the Fujisaki-Okamoto transformation, and finally ML-KEM, I would like to reflect on several aspects that I found particularly insightful.

6.4.1 Why IND-CCA Security Is Non-Trivial

A classic example that illustrates the difficulty of IND-CCA security is the El Gamal encryption scheme. El Gamal is IND-CPA secure under the DDH assumption, but it is *not* IND-CCA secure. Given a challenge ciphertext $(c_1, c_2) = (g^y, m \cdot h^y)$, an adversary can simply multiply c_2 by a known factor α and submit the modified ciphertext $(c_1, \alpha \cdot c_2)$ to the decryption oracle. The oracle returns $\alpha \cdot m$, from which the adversary recovers m directly. The attack works because El Gamal's ciphertexts are *malleable*.

The Cramer-Shoup KEM (CS3) closes this gap by introducing *redundancy and verification*. The decapsulation algorithm verifies that the ciphertext components satisfy a carefully designed algebraic relation involving $v = H(a, \hat{a})$. Any tampering that changes $(a, \hat{a})$ will change v and cause the verification to fail. The adversary cannot adjust d to satisfy the new relation without knowing the secret exponents.

The Fujisaki-Okamoto transformation achieves a similar effect but in a *generic* way. Instead of algebraic verification, FO uses *re-encryption*: the decapsulator recovers m', recomputes the ciphertext as $\text{Encrypt}(pk, m'; G(m'))$, and only accepts if it matches the received ciphertext. Both techniques are brilliant but illustrate different trade-offs: CS3 uses structured algebra under DDH, while FO uses generic re-encryption in the random oracle model.

6.4.2 The Power of Hybrid Games

Among all proof techniques encountered, the hybrid game sequence used to prove IND-CCA security of CS3 left the deepest impression on me. The proof proceeds through **seven games** ($\mathbf{G}_0$ to $\mathbf{G}_7$), each modifying one small aspect of the original security experiment.

What makes this sequence so impressive is how each step isolates a *single* cryptographic assumption or a *single* statistical artifact. Each transition is either:

- Conceptually identical (no change in distribution),
- Statistically close (bounded by a small term like $1/q$ or q_D/q), or
- Computationally close (bounded by an assumption like DDH or TCR).

When all transitions are combined, the total advantage is a *sum* of small terms. This taught me that even a complex security proof can be made transparent if the proof designer carefully

chooses the right intermediate game definitions. The art of security proofs lies largely in *finding the right sequence of hybrid games*.

6.4.3 Modularization as a Way of Thinking

One of the most valuable intellectual takeaways from this project is the power of modular abstraction.

First, **Shoup's KEM/DEM systematization** separates the concerns of asymmetric key establishment and symmetric data protection. If a KEM is IND-CCA secure and a DEM is IND-CCA secure, then their composition is automatically IND-CCA secure. This modularity mirrors how secure systems should be built.

Second, **the HHK modular analysis of FO** decomposes a complex transformation into simpler pieces T and U. T turns an OW-CPA PKE into a deterministic OW-PCA PKE; U turns an OW-PCVA PKE into an IND-CCA KEM. This decomposition clarified for me why FO can start from a very weak assumption (OW-CPA) and still achieve IND-CCA. Before reading HHK, I viewed FO as a single opaque recipe. Afterward, I saw it as a *framework*.

These two modularization efforts fundamentally changed how I think about cryptographic design: good cryptography is not just about hard problems, but about structuring protocols so that their security becomes analyzable and composable.

Chapter 7 Conclusion

The transition from traditional Public-Key Encryption (PKE) and interactive Diffie-Hellman paradigms to the Key Encapsulation Mechanism (KEM) represents one of the most profound architectural shifts in modern applied cryptography. This paper has traced the complete trajectory of KEM, demonstrating how a simple syntactic refinement—delegating the encapsulation of a random secret to the asymmetric primitive—has streamlined security proofs, enabled generic composition, and paved the way for the post-quantum era.

Through our technical review, we highlighted the critical milestones that matured KEM into a robust standard. The KEM/DEM systematization and the Cramer-Shoup construction established IND-CCA security as the uncompromising baseline for modern network protocols. Subsequently, the modular Fujisaki-Okamoto (FO) transformation provided the essential bridge for post-quantum cryptography, allowing inherently noisy and CPA-secure lattice-based primitives to achieve CCA security safely and efficiently.

To contextualize these theoretical constructs, we developed and evaluated jkem, a standard-compliant Rust implementation of ML-KEM. Our empirical evaluation emphasizes that the security of modern KEMs heavily relies not only on mathematical hardness assumptions but also on meticulous software engineering. Implementing constant-time decapsulation and correctly managing the implicit rejection paths are as crucial as the underlying Module-LWE problem itself. While portable software implementations offer excellent educational and structural clarity, our benchmarks reaffirm the necessity for architecture-specific optimizations to meet the stringent latency requirements of global-scale protocols.

Finally, our analysis of HPKE and Hybrid TLS 1.3 deployments underscores a fundamental reality: a KEM is a powerful building block, but it is not a standalone secure channel. The overarching protocols must rigorously enforce context binding, identity authentication, and downgrade resistance. As the cryptographic ecosystem migrates toward post-quantum resilience, the true success of KEM will be measured by our ability to seamlessly integrate these encapsulation mechanisms into hybrid architectures, balancing conservative security margins with computational efficiency. Ultimately, the KEM abstraction has proven to be the right interface at the right time, providing the foundational stability needed to secure tomorrow's digital infrastructure against quantum threats.

References

[1] DIFFIE W, HELLMAN M E. New directions in cryptography[G]//Democratizing cryptography: the work of Whitfield Diffie and Martin Hellman. 2022: 365-390.

[2] SHOUP V. A proposal for an ISO standard for public key encryption[J]. Cryptology ePrint Archive, 2001.

[3] BARNES R, BHARGAVAN K, LIPP B, et al. Hybrid Public Key Encryption: 9180 [R/OL]. RFC Editor. 2022. https://www.rfc-editor.org/rfc/rfc9180.

[4] National Institute of Standards and Technology. Module-Lattice-Based Key-Encapsulation Mechanism Standard: 203[R/OL]. U.S. Department of Commerce. 2024. https://doi.org/10.6028/NIST.FIPS.203.

[5] CRAMER R, SHOUP V. Design and analysis of practical public-key encryption schemes secure against adaptive chosen ciphertext attack[J]. SIAM Journal on Computing, 2003, 33(1): 167-226.

[6] FUJISAKI E, OKAMOTO T. Secure integration of asymmetric and symmetric encryption schemes[C]//Annual international cryptology conference. 1999: 537-554.

[7] HOFHEINZ D, HÖVELMANNS K, KILTZ E. A modular analysis of the Fujisaki-Okamoto transformation[C]//Theory of Cryptography Conference. 2017: 341-371.

[8] TARGHI E E, UNRUH D. Post-quantum security of the Fujisaki-Okamoto and OAEP transforms[C]//Theory of Cryptography Conference. 2016: 192-216.

[9] BERNSTEIN D J, CHOU T, LANGE T, et al. Classic McEliece: conservative code-based cryptography[J]. NIST submissions, 2017, 1(1): 1-25.

[10] ARAGON N, BARRETO P, BETTAIEB S, et al. BIKE: bit flipping key encapsulation [J]. 2022.

[11] MELCHOR C A, ARAGON N, BETTAIEB S, et al. Hamming quasi-cyclic (HQC)[J]. NIST PQC Round, 2018, 2(4): 13.

[12] BOS J, DUCAS L, KILTZ E, et al. CRYSTALS-Kyber: a CCA-secure module-lattice-based KEM[C]//2018 IEEE European symposium on security and privacy (EuroS&P). 2018: 353-367.

[13] RESCORLA E. The Transport Layer Security (TLS) Protocol Version 1.3: 8446[R/OL]. RFC Editor. 2018. https://www.rfc-editor.org/rfc/rfc8446.

[14] STEBILA D, FLUHRER S, GUERON S. Hybrid key exchange in TLS 1.3: draft-ietf-tls-

hybrid-design-16[R/OL]. Internet Engineering Task Force. 2025. https://datatracker.ietf.org/doc/draft-ietf-tls-hybrid-design/16/.

[15] KWIATKOWSKI K, KAMPANAKIS P, WESTERBAAN B, et al. Post-quantum hybrid ECDHE-MLKEM Key Agreement for TLSv1.3: draft-ietf-tls-ecdhe-mlkem-04[R/OL]. Internet Engineering Task Force. 2026. https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/04/.

[16] DOWLING B, FISCHLIN M, GÜNTHER F, et al. A cryptographic analysis of the TLS 1.3 handshake protocol[J]. Journal of Cryptology, 2021, 34(4): 37.

[17] DOWLING B, HANSEN T B, PATERSON K G. Many a mickle makes a muckle: A framework for provably quantum-secure hybrid key exchange[C]//International Conference on Post-Quantum Cryptography. 2020: 483-502.

[18] SIKERIDIS D, KAMPANAKIS P, DEVETSIKIOTIS M. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH[C]//Proceedings of the 16th International Conference on emerging Networking EXperiments and Technologies. 2020: 149-156.

[19] BERNSTEIN D J, BRUMLEY B B, CHEN M S, et al. {OpenSSLNTRU}: Faster post-quantum {TLS} key exchange[C]//31st USENIX security symposium (USENIX Security 22). 2022: 845-862.

Appendix A List of Abbreviations

Abbreviation	Description
AEAD	Authenticated Encryption with Associated Data
CCA	Chosen Ciphertext Attack
CPA	Chosen Plaintext Attack
DDH	Decisional Diffie-Hellman
DEM	Data Encapsulation Mechanism
DH	Diffie-Hellman
ECDHE	Elliptic Curve Diffie-Hellman Ephemeral
FO	Fujisaki-Okamoto (Transformation)
HPKE	Hybrid Public Key Encryption
IND-CCA	Indistinguishability under adaptive Chosen Ciphertext Attack
IND-CPA	Indistinguishability under Chosen Plaintext Attack
KAT	Known Answer Test
KDF	Key Derivation Function
KEM	Key Encapsulation Mechanism
LWE	Learning With Errors
ML-KEM	Module-Lattice-Based Key-Encapsulation Mechanism
ML-LWE	Module Learning With Errors
NIST	National Institute of Standards and Technology
NTT	Number Theoretic Transform
OW-CPA	One-Wayness under Chosen Plaintext Attack
OW-PCVA	One-Wayness under Plaintext and Validity Checking Attacks
PKE	Public-Key Encryption
PQC	Post-Quantum Cryptography
QROM	Quantum Random Oracle Model
ROM	Random Oracle Model
TLS	Transport Layer Security

Appendix B Author Contributions

Team Member	Student ID	Specific Contributions
Shenyu Wu	523031910465	Project topic conceptualization, design and implementation of the core experimental framework, and empirical performance analysis, drafting of the Experiment and Application chapter.
Yuezhao Ma	523031910684	Background literature review, drafting of the Abstract, introduction and conclusion, overall LaTeX formatting and structural unification of the manuscript, and preparation of the final presentation slides.
Shengyuan Huang	523031910102	In-depth theoretical analysis, drafting of the core technical chapters, and synthesis of KEM applications within modern cryptographic protocol deployments.